

👉 XAI: Rule Based Approaches

FAMAF - UNC

First Semester 2026



**eXplainable
Artificial Intelligence**

Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

<https://interpretable-ml-class.github.io/>

The Annals of Applied Statistics
2015, Vol. 9, No. 3, 1350–1371
DOI: [10.1214/15-AOAS848](https://doi.org/10.1214/15-AOAS848)
© Institute of Mathematical Statistics, 2015

INTERPRETABLE CLASSIFIERS USING RULES AND BAYESIAN ANALYSIS: BUILDING A BETTER STROKE PREDICTION MODEL

BY BENJAMIN LETHAM^{*,1}, CYNTHIA RUDIN^{*,1}, TYLER H. MCCORMICK^{†,2}
AND DAVID MADIGAN^{‡,3}

Massachusetts Institute of Technology^{}, University of Washington[†]
and Columbia University[‡]*

We aim to produce predictive models that are not only accurate, but are also interpretable to human experts. Our models are decision lists, which consist of a series of *if...then...* statements (e.g., *if high blood pressure, then stroke*) that discretize a high-dimensional,

(Letham et al. 2015)

Contributions ---

- Introducing a generative model called **Bayesian Rule Lists (BRL)**
 - ▶ Goal is to output a decision list (`if then else-lif`)
- Novel prior structure to **encourage sparsity**
 - ▶ “Sparse” means the model has few active elements
 - ▶ Instead of using all possible rules, BRL keeps only a few that are sufficiently expressive.
 - ▶ Truncated Poisson priors over list length (λ) and rule complexity (η) encourage *sparse*, *interpretable* decision lists.
- Predictive accuracy **on par** with top classic algorithms (RF, SVM, CHADS₂)

Decision List: Titanic Example ---

if male **and** adult **then** *survival probability* 21% (19%–23%)
else if 3rd class **then** *survival probability* 44% (38%–51%)
else if 1st class **then** *survival probability* 96% (92%–99%)
else *survival probability* 88% (82%–94%)

- This **is one of many** accurate and interpretable decision lists that can be learned from the data — BRL captures this uncertainty by maintaining a posterior distribution over all of them.
- **Each rule fires in order** — the first matching rule determines the prediction.
- A male adult in 1st class gets **21%**, not **96%**, because the first rule takes priority.
- The values in parentheses are **95% credible intervals** — the narrower, the more data behind that rule.

Implementing a Decision List in Python ---

```
>>> def predict_survival(male, adult, passenger_class):  
...     if male and adult:  
...         return 0.21, (0.19, 0.23)  
...     elif passenger_class == 3:  
...         return 0.44, (0.38, 0.51)  
...     elif passenger_class == 1:  
...         return 0.96, (0.92, 0.99)  
...     return 0.88, (0.82, 0.94)  
...  
>>> survival, confidence = predict_survival(True, False, 3)  
>>> survival  
0.44  
>>> confidence  
(0.38, 0.51)
```

Introduction: BRL ---

- New type of **balance** between accuracy, interpretability, and computation
- **What about using other similar models?**
 - ▶ Decision trees (CART)
 - ▶ They employ greedy construction methods
 - ▶ Not particularly computationally demanding but affects quality of solution – both accuracy and interpretability

Pre-mined Rules ---

- A major source of practical feasibility: **pre-mined rules**
 - ▶ Reduces model space
 - ▶ Complexity of problem depends on number of pre-mined rules
- As long as pre-mined set is expressive, **accurate decision list can be found** + smaller model space means **better generalization** (Vapnik, 1995)

🍷 Pre-mined Rules: Intuition ---



Minimum Support = 3

- So the final candidate antecedents passed to BRL would be: {2}, {3}, {4}, {2,3}, {2,4}, {3,4}.
- This is Apriori algorithm. FP-growth is a single pass algorithm (more efficient).

Preliminaries: Notation ---

- Training data $\{(x_i, y_i)\}_{i=1}^n$, $x_i \in \mathbb{R}^d$, $y_i \in \{1, \dots, L\}$

$$\mathbf{x} = (x_1, \dots, x_n) \quad \mathbf{y} = (y_1, \dots, y_n)$$

Two labels:

- 1 stroke
- 2 no stroke

🍷 Warning ---

💻 ⚠️ Warning for students with a background in computer science

- **Intuition:** Bayesians use probability distributions like programmers use libraries (`numpy`, `sklearn`). Instead of building new theory from scratch, they pick standard distributions that fit their problem and update them with data.
- **Formally:** You encode prior beliefs using these distributions *before* seeing data, then combine them with observations via Bayes' theorem to get the posterior. The 'library' gives you the mathematical machinery, but you still need to choose the right prior and let the data update it.

Bayesian Decision Lists ---

if a_1 then $y \sim \text{Multinomial}(\theta_1)$, $\theta_1 \sim \text{Dirichlet}(\alpha + \mathbf{N}_1)$
else if a_2 then $y \sim \text{Multinomial}(\theta_2)$, $\theta_2 \sim \text{Dirichlet}(\alpha + \mathbf{N}_2)$
:
else if a_m then $y \sim \text{Multinomial}(\theta_m)$, $\theta_m \sim \text{Dirichlet}(\alpha + \mathbf{N}_m)$
else $y \sim \text{Multinomial}(\theta_0)$, $\theta_0 \sim \text{Dirichlet}(\alpha + \mathbf{N}_0)$.

Where:

- a_j : Antecedent (the "if" condition) for rule j
- $y \sim \text{Multinomial}(\theta_j)$: The predicted label follows a multinomial distribution with probabilities θ_j
- $\theta_j \sim \text{Dirichlet}(\alpha + \mathbf{N}_j)$: The parameters θ_j themselves have a posterior distribution (Dirichlet) that combines the prior pseudocounts α with the observed data counts $\mathbf{N}_j$
- α : Prior pseudocounts (hyperparameter). Setting $\alpha = (1, 1, \dots, 1)$ gives a uniform prior with no class preference
- $\mathbf{N}_j = (N_{j,1}, \dots, N_{j,L})$: Counts of observations captured by rule j for each of the L classes

Preliminaries: Sampling from a multinomial ---

The code simulates throwing a 6-sided die 20 times, where each face has equal probability $1/6$.

```
>>> import numpy as np
>>> np.random.multinomial(20, [1/6]*6, size=1)
array([[4, 1, 7, 5, 2, 1]])
```

- $[1/6] * 6 = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6]$
- returns how many times each face appeared: means face 1 appeared 4 times, face 2 once, face 3 seven times, etc.

Parameters are probability values

- θ_j , in BRL, is a vector of probabilities, one per class. For example, $\theta_j = [0.6, 0.3, 0.1]$ for a 3-class problem.
- When you sample a label y from $\text{Multinomial}(\theta_j)$, you're drawing one outcome according to those probabilities.



Preliminaries: Dirichlet Distribution ---

- **Dirichlet samples from this simplex:** Drawing from $\text{Dirichlet}(\alpha_1, \alpha_2)$ gives probability vectors like $(0.6, 0.4)$ or $(0.3, 0.7)$
- **Probability simplex:** The space of all valid probability vectors (sum to 1, non-negative)
 - ▶ Example: $(0.6, 0.4)$ is valid; $(0.6, 0.5)$ is not (sums to 1.1)
- **K-dimensional Dirichlet has k parameters $(\alpha_1, \alpha_2, \dots, \alpha_m)$**
 - ▶ Each α_k controls expected probability mass for dimension k
 - ▶ Higher $\alpha_k \rightarrow$ more weight on that dimension
 - ▶ All $\alpha_k = 1 \rightarrow$ uniform distribution over the simplex

$$\Theta \sim \text{Dirichlet}(\alpha_1, \alpha_2, \dots, \alpha_m)$$

$$P(\theta_1, \theta_2, \dots, \theta_m) = \frac{\Gamma(\sum_k \alpha_k)}{\prod_k \Gamma(\alpha_k)} \prod_{k=1}^m \theta_k^{\alpha_k - 1}$$

🍷 Dirichlet as Conjugate Prior for Multinomial ---

- **Conjugate prior:** A prior is conjugate when the posterior has the same distributional form as the prior — no complex integrals needed.

- **Prior:**

$$(p_1, \dots, p_k) \sim \text{Dirichlet}(\alpha_1, \dots, \alpha_k)$$

- **Posterior:** Just add observed counts to prior pseudocounts:

$$(p_1, \dots, p_k) \mid (x_1, \dots, x_k) \sim \text{Dirichlet}(\alpha_1 + x_1, \dots, \alpha_k + x_k)$$

🍷 Dirichlet as Conjugate Prior for Multinomial - Example ---

Prior

- **Prior** $\alpha = (1, 1, 1)$ (uniform, no class preference)
- + observed counts (5, 2, 3)
- **Posterior** = Dirichlet(6, 3, 4)

Why it matters for BRL:

Each rule's posterior is computed by simply adding observation counts to prior pseudocounts — clean and efficient.

Bayesian Association Rules ---

- A **Bayesian association rule** $a \rightarrow y$ predicts a label distribution instead of a single label:

$$a \rightarrow y \sim \text{Multinomial}(\theta)$$

- The parameters θ themselves are uncertain — we put a Dirichlet prior on them:

$$\theta \mid \alpha \sim \text{Dirichlet}(\alpha)$$

- After observing data $(\mathbf{x}, \mathbf{y})$, the posterior is simply:

$$\theta \mid \mathbf{x}, \mathbf{y}, \alpha \sim \text{Dirichlet}(\alpha + N)$$

- where $N = (N_{\cdot,1}, \dots, N_{\cdot,L})$ are the observed counts per class for observations captured by this rule.

Bayesian Association Rules ---

$$a \rightarrow y \sim \text{Multinomial}(\theta). \quad \theta | \alpha \sim \text{Dirichlet}(\alpha).$$

$$\theta | \mathbf{x}, \mathbf{y}, \alpha \sim \text{Dirichlet}(\alpha + N)$$

$$N = (N_{\cdot,1}, \dots, N_{\cdot,L})$$

Example

- Rule "age > 60 AND diabetes"
- Captures 10 patients: 7 with stroke, 3 without.
- With $\alpha = (1, 1)$,
- posterior = $\text{Dirichlet}(8, 4)$
- $\rightarrow$ estimated stroke probability = $8/12 \approx 67\%$

🍲 Finally ---

At this point we know the ingredients:

- What a **Bayesian Rule List (BRL)** is: an ordered `if-then-else` decision list with probabilistic predictions
- How to mine a collection of candidate rules using **FP-Growth** (frequent itemset mining)
- How a **single rule** combines observations with a Dirichlet-Multinomial model to estimate class probabilities

Now it's time to put it all together and **learn the full BRL from data**.

Generative Model 1/2 ---

- Sample a decision list length $m \sim p(m|\lambda)$.
- Sample the default rule parameter $\theta_0 \sim \text{Dirichlet}(\alpha)$.
- For decision list rule $j = 1, \dots, m$:
 - Sample the cardinality of antecedent a_j in d as $c_j \sim p(c_j|c_{<j}, \mathcal{A}, \eta)$.
 - Sample a_j of cardinality c_j from $p(a_j|a_{<j}, c_j, \mathcal{A})$.
 - Sample rule consequent parameter $\theta_j \sim \text{Dirichlet}(\alpha)$.
- For observation $i = 1, \dots, n$:
 - Find the antecedent a_j in d that is the first that applies to x_i .
 - If no antecedents in d apply, set $j = 0$.
 - Sample $y_i \sim \text{Multinomial}(\theta_j)$.

Our goal instead of finding a single optimal list like a greedy algorithm, BRL assigns a **probability to every possible list** — giving us the full posterior distribution over decision lists.

$$p(d|\mathbf{x}, \mathbf{y}, \mathcal{A}, \alpha, \lambda, \eta) \propto p(\mathbf{y}|\mathbf{x}, d, \alpha)p(d|\mathcal{A}, \lambda, \eta).$$

continue...

Generative Model 2/2 ---

$$p(d|\mathbf{x}, \mathbf{y}, \mathcal{A}, \alpha, \lambda, \eta) \propto p(\mathbf{y}|\mathbf{x}, d, \alpha)p(d|\mathcal{A}, \lambda, \eta).$$

The posterior probability of list d , given the data $\mathbf{x}$, labels $\mathbf{y}$, candidate rules $\mathcal{A}$, and hyperparameters α (*pseudocounts*), λ, η , is proportional to the product of:

- $p(\mathbf{y}|\mathbf{x}, d, \alpha)$: the **likelihood** — how well list d explains the observed labels
- $p(d|\mathcal{A}, \lambda, \eta)$: the **prior** — how probable that list was before seeing any data
- λ controls the prior expected list length (number of **if then**),
- η controls the prior expected antecedent cardinality (number of conditions per rule).

***In short:** posterior $\propto$ likelihood $\times$ prior

continue...

🍷 Prior Probabilities ---

$$p(d|\mathcal{A}, \lambda, \eta) = p(m|\mathcal{A}, \lambda) \prod_{j=1}^m p(c_j|c_{<j}, \mathcal{A}, \eta) p(a_j|a_{<j}, c_j, \mathcal{A}).$$

Truncated Poisson:

$$p(m|\mathcal{A}, \lambda) = \frac{(\lambda^m/m!)}{\sum_{j=0}^{|\mathcal{A}|} (\lambda^j/j!)}, \quad m = 0, \dots, |\mathcal{A}|.$$

Ensures that sampled values are within bounds!

Also, ensures expected value is close to λ when there are a large number of pre-mined rules

🍷 Prior Probabilities ---

Another Truncated Poisson,

$$p(c_j | c_{<j}, \mathcal{A}, \eta) = \frac{(\eta^{c_j} / c_j!)}{\sum_{k \in R_{j-1}(c_{<j}, \mathcal{A})} (\eta^k / k!)}, \quad c_j \in R_{j-1}(c_{<j}, \mathcal{A}).$$

$p(a_j | a_{<j}, c_j, \mathcal{A})$ is sampled uniformly from available antecedents with appropriate cardinality.

Likelihood ---

- Likelihood is the product of multinomial probability mass functions for the observed label counts at each rule

$$p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\theta}) = \prod_{j: \sum_l N_{j,l} > 0} \text{Multinomial}(\mathbf{N}_j | \theta_j),$$

$$\theta_j \sim \text{Dirichlet}(\boldsymbol{\alpha}).$$

Marginalize over θ_j , integrate out the intermediate parameter θ_j

Markov Chain Monte Carlo ---

- Generate a chain of random samples until convergence
- Each random sample is a stepping stone for the next one (chain)
- New samples do not depend on any samples before the previous one (Markov)

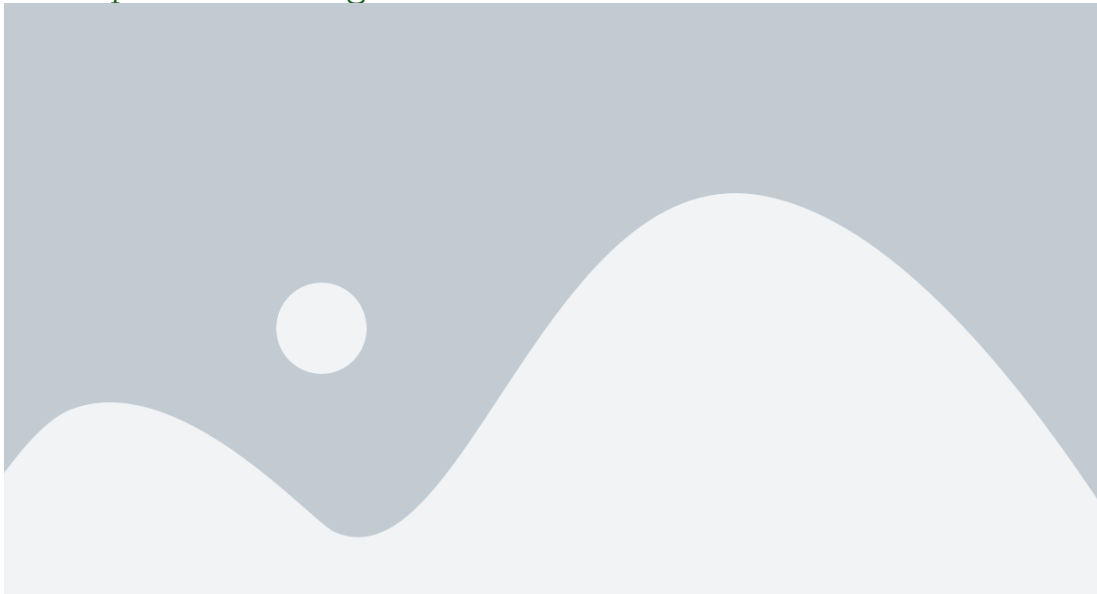
Markov Chain Monte Carlo ---

- How to go to (optimal) d^* from current d^t
- Move an antecedent to a different position in the list
- Add an antecedent that is not currently in the list
- Remove an antecedent from the list

Metropolis Hastings ---

- Start with a random decision list
- Choose a move based on “proposal distribution” Q
- After you choose your move, you compute an acceptance probability A
- Generate a random number u
- If $u \leq A$, then accept; otherwise reject

🍷 Metropolis Hastings ---



🍷 Proposal Probabilities ---

- Move chosen uniformly
- Which antecedents and their new position is also chosen uniformly



🍷 Estimating label of a new observation ---

$$p(\tilde{y} = l | \tilde{x}, d, \mathbf{x}, \mathbf{y}, \alpha) = \frac{\alpha_l + N_{j(d, \tilde{x}), l}}{\sum_{k=1}^L (\alpha_k + N_{j(d, \tilde{x}), k})}.$$

Match the antecedent by looking at feature values of new observation

Experiments ---



Tic-Tac-Toe ---



Stroke Prediction ---

- $N = 12,586$, 14% had stroke
- 6000 times larger than data for CHADS2 score
- Pre-mining: support 10% and max cardinality 2
- 5 fold evaluation

🍷 Stroke Prediction ---



🍷 Stroke Prediction ---



🍷 Stroke Prediction - AUC ---



Paper 2 ---



Contributions ---

- A framework called **Interpretable Decision Sets** (IDS) for classification
- **Novel objective function** + proof of **submodularity**
- Optimization procedure with optimality guarantees
- **Detailed metrics for evaluating interpretability** + user studies

Motivation ---

- Traditional classification models optimize for predictive accuracy
- Very little understanding of the model itself and its predictions
- Model being “readable” is not enough
- **Humans should be able to reason about predictions and readily explain the functionality of the model**

Decision Lists vs Decision Sets ---

Decision Lists

- Ordered sequence of if-then-else rules
- Order matters: first matching rule applies
- Like a chain of if/else-if statements

Decision Sets

- Unordered collection of if-then rules
- Each rule independently assigns a class
- Rules can overlap (multiple rules may fire)

🍷 Decision Sets ---



🍷 Criteria for Interpretability ---

- **Parsimony**: Fewer rules with fewer conditions
 - ▶ Cognitive limits of human understanding
- **Distinctness**: Minimal overlap of rules w.r.t the data points they cover
 - ▶ No redundant and contradicting explanations of data points
- **Class Coverage**: Explain all the classes in the data
 - ▶ Rules explaining minority classes are important

Problem Formulation ---



Desiderata ---

- We need to optimize for the following criteria
 - ▶ Recall
 - ▶ Precision
 - ▶ Distinctness
 - ▶ Parsimony
 - ▶ Class Coverage
- Recall and Precision $\Rightarrow$ Accurate predictions
- Distinctness, Parsimony, and Class Coverage $\Rightarrow$ Interpretability

Objective Function ---

Parsimony

- Fewer rules: $f_1(\mathcal{R}) = |\mathcal{S}| - \text{size}(\mathcal{R})$
- Fewer predicates: $f_2(\mathcal{R}) = L_{\max} \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} \text{length}(r)$

Objective Function ---

Distinctness

- Intra-class overlap:

$$f_3(\mathcal{R}) = N \cdot |S|^2 - \sum_{\substack{r_i, r_j \in \mathcal{R} \\ i \leq j \\ c_i = c_j}} \text{overlap}(r_i, r_j)$$

- Inter-class overlap:

$$f_4(\mathcal{R}) = N \cdot |S|^2 - \sum_{\substack{r_i, r_j \in \mathcal{R} \\ i \leq j \\ c_i \neq c_j}} \text{overlap}(r_i, r_j)$$

🍷 Objective Function ---

Class Coverage

$$f_5(\mathcal{R}) = \sum_{c' \in \mathcal{C}} \mathbf{1} (\exists r = (s, c) \in \mathcal{R} \text{ such that } c = c')$$

Check if there exists some rule corresponding to a given class c

🍷 Objective Function ---

Precision

- Minimize “incorrect” covers:

$$f_6(\mathcal{R}) = N \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} |\text{incorrect-cover}(r)|$$

Given a rule $r = (s, c)$, the number of data points which satisfy s but do not belong to class c .

Objective Function ---

Recall

- Encourage at least one “correct” cover per data point:

$$f_7(\mathcal{R}) = \sum_{(\mathbf{x}, y) \in \mathcal{D}} \mathbf{1}(|\{r | (\mathbf{x}, y) \in \text{correct-cover}(r)\}| \geq 1)$$

Given a rule $r = (s, c)$, the number of data points which satisfy s and belong to class c .

🍷 Objective Function ---

- Complete objective is

$$\operatorname{argmax}_{\mathcal{R} \subseteq \mathcal{S} \times \mathcal{C}} \sum_{i=1}^7 \lambda_i f_i(\mathcal{R})$$

- The intra-class and inter-class overlap terms are non-monotone
- The parsimony, overlap, and precision terms are non-normal
- All the component terms are submodular

Submodularity ---

Diminishing returns characterization

$$F(A \cup d) - F(A) \geq F(B \cup d) - F(B)$$

Gain of adding d to a small set

Gain of adding d to a large set

Key Property

A non-negative linear combination of submodular functions is submodular

🍷 Objective Function ---

- Complete objective is

$$\operatorname{argmax}_{\mathcal{R} \subseteq \mathcal{S} \times \mathcal{C}} \sum_{i=1}^7 \lambda_i f_i(\mathcal{R})$$

Result

The complete objective is non-negative, non-normal, non-monotone, submodular

🍷 Optimizing the Objective ---

- Maximizing a non-monotone submodular function is **NP-hard**
- **Smooth local search** [SLS] algorithm provides a $2/5$ approximation [Feige, Mirrokni, Vondrak FOCS 07; SIAM Comp. J. 11]
 - ▶ Will be at least $2/5$ of the optimal solution

🍷 Submodular Maximization: Local Search ---



🍷 Submodular Maximization: Local Search ---



🍷 Submodular Maximization: Local Search ---



🍷 Submodular Maximization: Local Search ---



🍷 Submodular Maximization: Local Search ---



🍷 Submodular Maximization: Local Search ---



🍷 Submodular Maximization: Local Search ---



Local Search ---

- $\sim 1/3$ approximation
 - ▶ At least $1/3$ of optimal solution
- we use a slightly different version of this algorithm
 - ▶ Smooth local search
 - ▶ $2/5$ approximation

🍷 Smooth Local Search ---



Evaluation: Datasets ---



🍷 Evaluating Predictive Performance ---



🍷 Evaluating Goodness of Rules ---

- Results on Medical Diagnosis Data



🍷 Ablation Study ---

- Results on Medical Diagnosis Data



Thank You!



References --- |

- Lakkaraju, Himabindu, Stephen H. Bach, and Jure Leskovec. 2016. “Interpretable Decision Sets: A Joint Framework for Description and Prediction.” In *Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining*, 1675–84. ACM.
- Letham, Benjamin, Cynthia Rudin, Tyler H. McCormick, and David Madigan. 2015. “Interpretable Classifiers Using Rules and Bayesian Analysis: Building a Better Stroke Prediction Model.” *The Annals of Applied Statistics* 9 (3): 1350–71.